

# Stack recomendado

## FRONTEND

### Framework de React

→ **Vite + React**

*Rápido, moderno, liviano, mejor DX que CRA.*

---

### Enrutamiento

→ **TanStack Router**

*Más moderno, typesafe, mejor performance que React Router.*

---

### Estado del cliente

→ **Zustand**

*Para estado global simple y moderno.*

---

### Manejo de llamadas a API

→ **TanStack Query**

*Se complementa con fetch/axios.*

### Cache / Server state

→ **TanStack Query**

*Cache, revalidación, mutations, sincronización.*

*(Redis no aplica en frontend.)*

---

### Librería de componentes

→ **ShadCN/UI (versión para Vite)**

*Actualmente el estándar moderno de componentes + Tailwind.*

---

## Sistema de estilos

→ **TailwindCSS**

*El setup ideal con Vite y ShadCN.*

---

## Manejo de formularios

→ **React Hook Form + Zod**

*Validación tipada y excelente performance.*

---

## Runtime de Node (para dev/build)

→ **Node 20 LTS**

*Más estable que 18, más maduro que 22.*

---

## Internacionalización

→ **i18next + react-i18next**

*La opción madura para React sin framework.*

---

## Testing

→ **Vitest + React Testing Library + Playwright (E2E)**

*Moderno y rápido.*

---

## Herramientas de desarrollo

→ **ESLint + Prettier + Husky + lint-staged**

*El combo estándar actual.*

---

## Monitoreo de frontend

→ **Azure Monitor + Azure Application Insights**

**Empieza con OpenReplay** instalado en Azure (una VM o un contenedor) o Microsoft Clarity

App Insights JS SDK o Sentry Free Tier

---

## Variables de entorno

→ **dotenv + .env (o .envrc si usas direnv)**

*(Vite los expone con `VITE_`.)*

---

## Git branching model

→ **GitHub Flow Development**

*(Moderno, simple, ideal para equipos ágiles.)*

---

## Monorepo

→ **Sí, con Turborepo**

*Solo si tendrás frontend + API + libs compartidas.*

---

## CI/CD

→ **GitHub Actions**

*Más moderno, flexible y popular.*

---

## Gestor de paquetes

→ **pnpm**

*Más rápido, más eficiente y estándar moderno 2024–2025.*

---

## API

### Runtime

→ **Node 20 LTS**

---

### Framework

→ **Fastify**

*Más moderno, performante y tipado que Express.*

---

## Lenguaje

→ **TypeScript** (obvio)

---

## Arquitectura

→ **Modular Monolith + Clean Architecture** dentro de cada módulo

*Independencia, testabilidad, mantenibilidad.*

---

## Ruteo

→ **Fastify routes + Controllers + Handlers**

*Simple y escalable.*

---

## Validación de Request

→ **Zod**

*Tipo-safe, moderno, DX increíble.*

---

## ORM / Base de datos

→ **Prisma**

*El estándar moderno, excelente con TypeScript.*

(Opción alternativa moderna: **Drizzle ORM**.)

---

## Seguridad

→ CORS + Helmet

→ Rate limiting (Fastify-rate-limit)

→ Sanitización de inputs

---

## Variables de entorno

→ **dotenv + zod** para validarlas

*(O Doppler si quieres un Secrets Manager real.)*

---

## Manejo de errores

→ **Middleware global + Fastify error handler + códigos HTTP tipados**

---

## Logging

→ **Pino**

*El más rápido, nativo en Fastify.*

---

## Testing

→ **Vitest + Supertest**

*Moderno + ligero.*

*(Opcional: Pact para contract testing.)*

---

## Performance

→ **Redis como caché**

→ **BullMQ para colas** (si necesitas jobs)

→ **Fastify plugins de performance**

---

## Documentación de API

→ **OpenAPI (Swagger) autogenerado con fastify-swagger**

*Estándar universal.*

---

## Observabilidad

→ **Azure Application Insights**

## OpenTelemetry

*El estándar 2025.*

---

## CI/CD

→ **GitHub Actions**

*(Mejor integración y soporte.)*

---

## Gestor de paquetes

→ **pnpm**

## Infraestructura

### Proveedor Cloud

→ **Azure**

---

### laC engine

→ **Bicep** (nativo Azure, declarativo)

*perfecto para Azure, gratis, simple*

---

### Estructura laC

→ Módulos por dominio (network, app, monitor, redis, db)

*escalable y limpio*

---

### Gestión de dependencias

→ Outputs entre módulos

---

### Variables laC

→ .parameters.json + azd env

*forma recomendada por Microsoft*

---

### Secrets

➡ Azure Key Vault (referenciado desde Bicep)

*seguro y barato.*

---

## Entornos

➡ `infra/dev` , `infra/staging` , `infra/prod`

*aislados, controlados*

---

## Pipeline IaC

➡ GitHub Actions con `azd up` + approvals

*moderno, simple.*

- ¿Cuándo se aplica IaC?
    - on merge a main
    - on tag
    - manual approval para prod
  - ¿Qué estrategia de despliegue?
    - `plan` + `apply`
    - pipelines separados
  - ¿Tu IaC se aplicará automáticamente o necesitará aprobación humana?
  - ¿Cómo combinarás IaC + aplicación?
    - IaC primero, app después
    - App deploy (Azure Container Apps) con ambiente ya creado
    - azd hace todo junto (infra + app deploy)
  - ¿IaC y app serán parte del mismo pipeline (end-to-end) o pasos independientes?
- 

## Recursos base

➡ Azure Container Apps + Log Analytics + App Insights + Redis

*tu stack ideal*

---

## **Monitoreo**

→ Azure Monitor

*métricas básicas gratis o muy baratas.*

---

## **Networking**

→ VNet opcional (depende del costo)

*flexible según presupuesto*